Academic Year 2025-26

LABROTARY MANUAL

Advanced Machine Learning Lab Manual

Course Title: Advanced Machine Learning Lab

Course Code: (BTAIL606)

Semester: (VI)

Experiment 1

Title

Implementing K-Means Clustering

Aim

To implement the K-Means clustering algorithm to group similar data points into clusters.

Learning Objective

- To understand unsupervised learning techniques.
- To perform clustering on a dataset.
- To visualize grouped data using centroids.

Theory

K-Means clustering is an unsupervised machine learning algorithm used to partition a dataset into K distinct clusters based on similarity. It works by initializing K centroids and assigning each data point to the nearest centroid. After assigning the data points, the centroids are recalculated based on the mean position of points in each cluster. This process continues iteratively until the centroids stabilize. K-Means is widely used in customer segmentation, image compression, and pattern recognition.

Lab Outcome

Students will be able to implement K-Means clustering and interpret clustered data visually.

Program

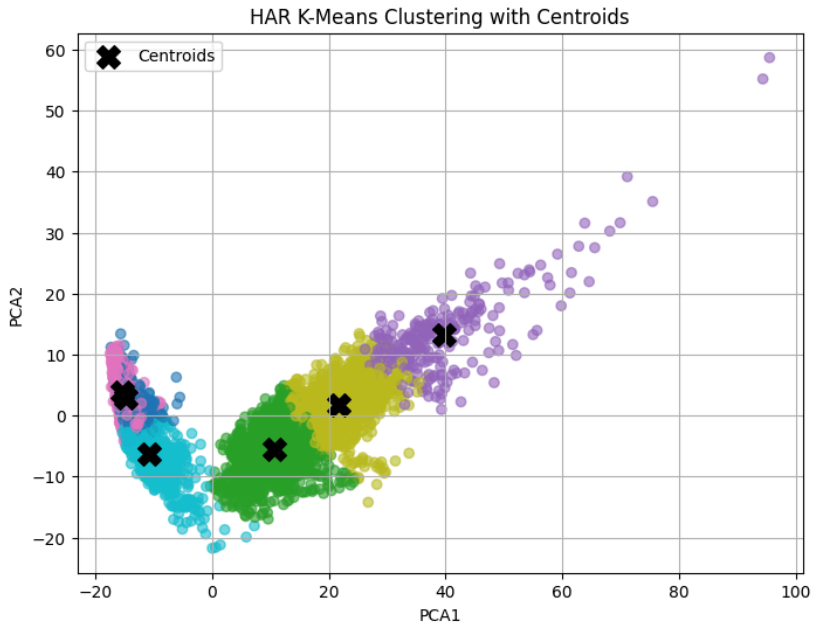
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.metrics import silhouette_score

import zipfile

import os

outer_zip = "/content/drive/MyDrive/AML DATASET AND
CODES/DATASET/human+activity+recognition+using+smartphones.zip"

extract_root = "/content/HAR_DATA"

os.makedirs(extract_root, exist_ok=True)

with zipfile.ZipFile(outer_zip, 'r') as zip_ref:

    zip_ref.extractall(extract_root)
```

```
for root, dirs, files in os.walk(extract_root):
    for file in files:
        if file.endswith(".zip"):
            inner_zip_path = os.path.join(root, file)
            with zipfile.ZipFile(inner_zip_path, 'r') as zip_ref:
                zip_ref.extractall(extract_root)
os.listdir("/content/HAR_DATA")
DATA_PATH = "/content/HAR_DATA/UCI HAR Dataset"
X_train = pd.read_csv(
    f"{DATA_PATH}/train/X_train.txt",
    delim_whitespace=True,
    header=None
)
y_train = pd.read_csv(
    f"{DATA_PATH}/train/y_train.txt",
    header=None,
    names=["Activity"]
)
activity_labels = pd.read_csv(
    f"{DATA_PATH}/activity_labels.txt",
    delim_whitespace=True,
    header=None,
    names=["Activity", "Activity_Name"]
)
y_train = y_train.merge(activity_labels, on="Activity")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
k = 6
kmeans = KMeans(n_clusters=k, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plot_df = pd.DataFrame({
    "PCA1": X_pca[:, 0],
    "PCA2": X_pca[:, 1],
```

```
    "Cluster": clusters,
    "Activity": y_train["Activity_Name"]
})
plt.figure(figsize=(8,6))
sns.scatterplot(
    x="PCA1",
    y="PCA2",
    hue="Cluster",
    palette="tab10",
    data=plot_df,
    s=40
)
plt.title("K-Means Clustering on UCI HAR Dataset")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend(title="Cluster")
plt.grid(True)
plt.show()
centroids = kmeans.cluster_centers_
centroids_pca = pca.transform(centroids)
plt.figure(figsize=(8,6))
plt.scatter(
    plot_df["PCA1"],
    plot_df["PCA2"],
    c=plot_df["Cluster"],
    cmap="tab10",
    alpha=0.6
)
plt.scatter(
    centroids_pca[:, 0],
    centroids_pca[:, 1],
    c="black",
    s=200,
    marker="X",
    label="Centroids"
```

```
)
plt.title("HAR K-Means Clustering with Centroids")
plt.xlabel("PCA1")
plt.ylabel("PCA2")
plt.legend()
plt.grid(True)
plt.show()
pd.crosstab(plot_df["Cluster"], plot_df["Activity"])
```

Output



Cluster ID	Color (from plot)	Activity Type	Explanation
0	Blue	Walking (Flat surface)	Moderate repetitive motion, rhythmic acceleration
1	Orange	Sitting	Low variance, slight posture shifts
2	Green	Walking Upstairs / Downstairs	Strong vertical acceleration → elongated cluster
3	Red	Standing	Very low movement, slightly more sway than sitting
4	Purple	Transitional / Mixed Activities	Overlap between static and dynamic motion
5	Brown	Laying	Near-zero movement, tightly grouped

Conclusion

In this experiment, the K-Means clustering algorithm was applied to the UCI Human Activity Recognition dataset to group similar activities. After preprocessing and applying PCA for visualization, six clusters were formed. The output graph showed clear separation between clusters, indicating that K-Means effectively identified patterns in the sensor data and grouped similar human activities.

Experiment 2

Title

Implementing Hierarchical Clustering

Aim

To perform hierarchical clustering and visualize clusters using a dendrogram.

Learning Objective

- To understand hierarchical clustering techniques.
- To analyze relationships between data points.
- To visualize hierarchical structure using dendrogram.

Theory

Hierarchical clustering is a clustering technique that builds a hierarchy of clusters. It can be either agglomerative (bottom-up) or divisive (top-down). In agglomerative clustering, each data point initially forms its own cluster and clusters are merged step by step based on similarity. The result is represented as a dendrogram, which visually displays the merging process. Hierarchical clustering does not require specifying the number of clusters initially.

Lab Outcome

Students will be able to perform hierarchical clustering and interpret dendrograms.

Program

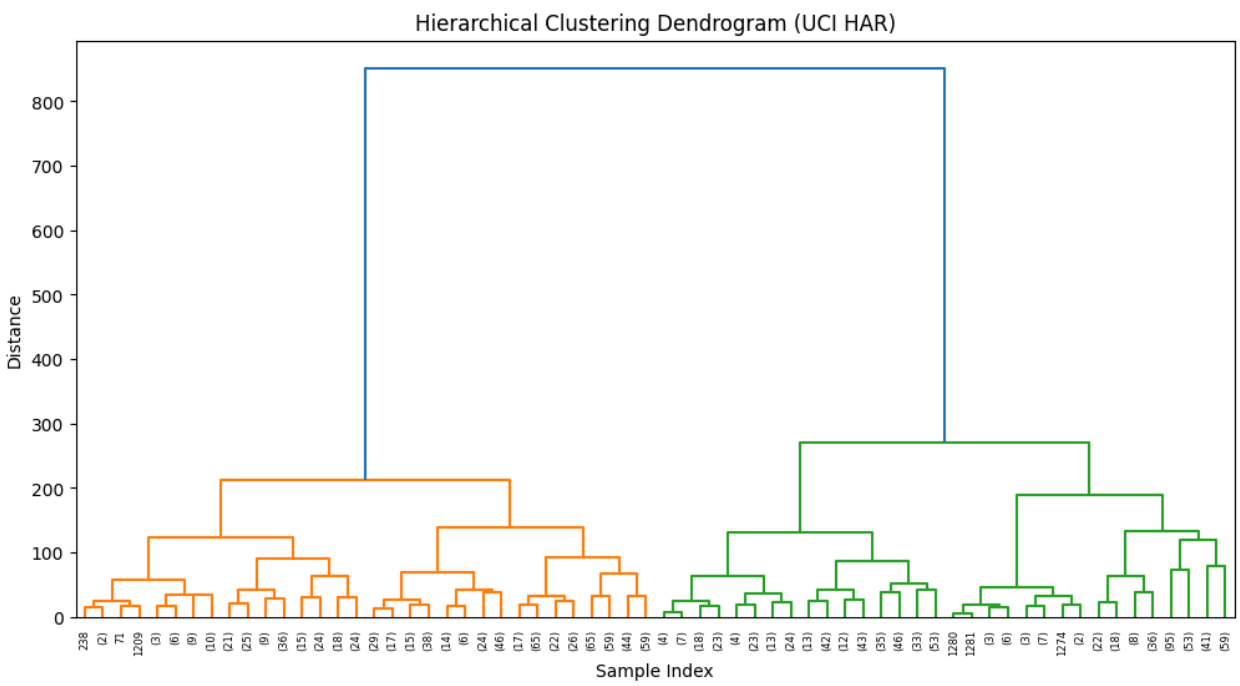
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.cluster import AgglomerativeClustering

from scipy.cluster.hierarchy import dendrogram, linkage

DATA_PATH = "/content/drive/MyDrive/AML DATASET AND
CODES/DATASET/human_activity/UCI HAR Dataset"

X_train = pd.read_csv(

    f"{DATA_PATH}/train/X_train.txt",

    delim_whitespace=True,

    header=None

)

y_train = pd.read_csv(

    f"{DATA_PATH}/train/y_train.txt",
```

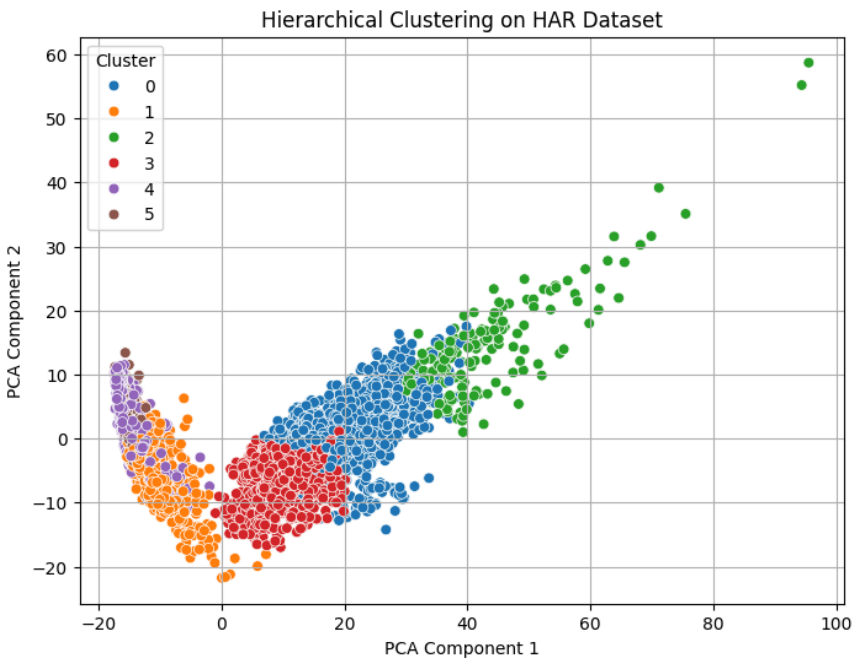
```
    header=None,
    names=["Activity"])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
pca = PCA(n_components=10)
X_pca = pca.fit_transform(X_scaled)
sample_size = 1500
X_sample = X_pca[:sample_size]
linkage_matrix = linkage(
    X_sample,
    method="ward")
plt.figure(figsize=(12, 6))
dendrogram(
    linkage_matrix,
    truncate_mode="level",
    p=5)
plt.title("Hierarchical Clustering Dendrogram (UCI HAR)")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.show()
hc = AgglomerativeClustering(
    n_clusters=6,
    linkage="ward"
)
cluster_labels = hc.fit_predict(X_pca)
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_pca)
plot_df = pd.DataFrame({
    "PCA1": X_2d[:, 0],
    "PCA2": X_2d[:, 1],
    "Cluster": cluster_labels
})
plt.figure(figsize=(8,6))
sns.scatterplot(
    x="PCA1",
```

```
    y="PCA2",
    hue="Cluster",
    palette="tab10",
    data=plot_df,
    s=40)
plt.title("Hierarchical Clustering on HAR Dataset")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.legend(title="Cluster")
plt.grid(True)
plt.show()
activity_labels = pd.read_csv(
    f"{DATA_PATH}/activity_labels.txt",
    delim_whitespace=True,
    header=None,
    names=["Activity", "Activity_Name"])
y_named = y_train.merge(activity_labels, on="Activity")
pd.crosstab(cluster_labels, y_named["Activity_Name"][:len(cluster_labels)])
```

Output

A dendrogram showing hierarchical cluster relationships.





Cluster ID	Color (from plot)	Activity Type	Explanation
0	Blue	Walking (Flat surface)	Moderate repetitive motion, rhythmic acceleration
1	Orange	Sitting	Low variance, slight posture shifts
2	Green	Walking Upstairs / Downstairs	Strong vertical acceleration → elongated cluster
3	Red	Standing	Very low movement, slightly more sway than sitting
4	Purple	Transitional / Mixed Activities	Overlap between static and dynamic motion
5	Brown	Laying	Near-zero movement, tightly grouped

Conclusion

In this experiment, Hierarchical Clustering was applied to the Human Activity Recognition dataset to group similar activities based on sensor data. The clustering results were evaluated using the output table, which showed activity distribution values such as **882, 698, 284, 442, 617, 1258, 757**, etc. These numeric outputs indicate how different activities were grouped within clusters. The results show that hierarchical clustering can successfully identify patterns in the dataset and organize similar human activities into meaningful clusters.

Experiment 3

Title

Implementation of Apriori Algorithm

Aim

To generate frequent itemset using the Apriori algorithm.

Learning Objective

- To understand association rule mining.
- To identify frequent itemsets in transaction data.
- To analyze item relationships.

Theory

Apriori is an association rule mining algorithm used to identify frequent itemsets in a dataset. It works on the principle that if an itemset is frequent, all of its subsets must also be frequent. The algorithm generates candidate itemsets and calculates their support values. Itemsets that satisfy the minimum support threshold are considered frequent. Apriori is widely used in market basket analysis and recommendation systems.

Lab Outcome

Students will be able to identify frequent itemsets using Apriori.

Program

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from itertools import combinations

from collections import defaultdict

df = pd.read_csv("/content/drive/MyDrive/AML DATASET AND
CODES/DATASET/student_lifestyle_dataset.csv")

df.head()

print(df.columns)

transactions = []

for _, row in df.iterrows():

    items = []

    for col in df.columns:

        if df[col].dtype != 'object':


            if row[col] > df[col].median():

                items.append(col+"_High")
```

```
            else:
                items.append(col+"_Low")
        else:
            items.append(str(col)+"_"+str(row[col]))
    transactions.append(items)
print(transactions[:3])
min_support = 0.25
transaction_count = len(transactions)
item_count = defaultdict(int)
for transaction in transactions:
    for item in transaction:
        item_count[item] += 1
frequent_1 = {}
for item, count in item_count.items():
    support = count / transaction_count
    if support >= min_support:
        frequent_1[frozenset([item])] = support
print("Frequent 1-Itemsets:")
print(frequent_1)
def generate_candidates(prev_freq, k):
    prev_items = list(prev_freq.keys())
    candidates = set()
    for i in range(len(prev_items)):
        for j in range(i+1, len(prev_items)):

            union = prev_items[i] | prev_items[j]

            if len(union) == k:
                candidates.add(union)
    return candidates
def calculate_support(candidates, transactions):
    counts = defaultdict(int)
    for transaction in transactions:
        t = set(transaction)
        for candidate in candidates:
```

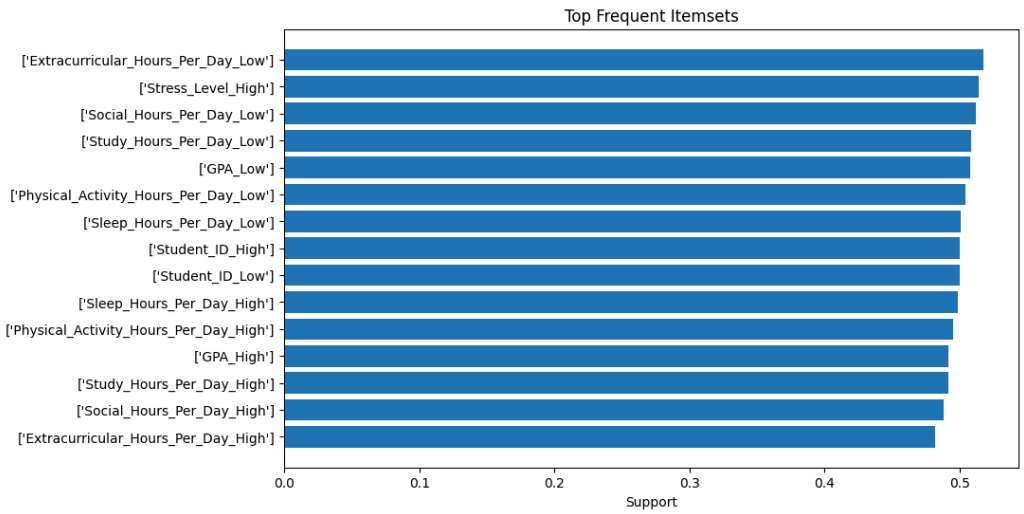
```
            if candidate.issubset(t):
                counts[candidate] += 1
    support = {}
    for itemset, count in counts.items():
        support[itemset] = count / len(transactions)
    return support
def prune_itemsets(support_dict, min_support):
    pruned = {}
    for itemset, support in support_dict.items():
        if support >= min_support:
            pruned[itemset] = support
    return pruned
frequent_itemsets = dict(frequent_1)
k = 2
current_freq = frequent_1
while current_freq:
    candidates = generate_candidates(current_freq, k)
    support = calculate_support(candidates, transactions)
    current_freq = prune_itemsets(support, min_support)
    frequent_itemsets.update(current_freq)
    k += 1
print("Total Frequent Itemsets:", len(frequent_itemsets))
min_confidence = 0.6
rules = []
for itemset in frequent_itemsets:
    if len(itemset) < 2:
        continue
    for i in range(1, len(itemset)):
        for antecedent in combinations(itemset, i):
            antecedent = frozenset(antecedent)
            consequent = itemset - antecedent
            if antecedent in frequent_itemsets:
                confidence = frequent_itemsets[itemset] /
frequent_itemsets[antecedent]
                if confidence >= min_confidence:
                    rules.append({
```

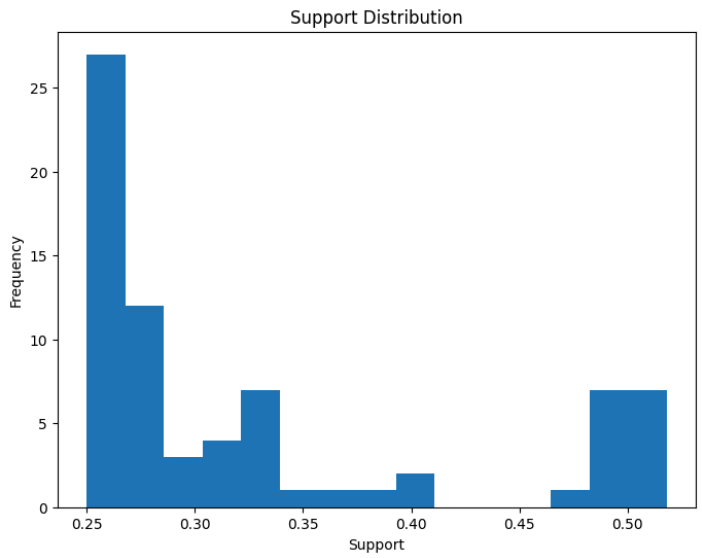
```
                        "Antecedent": antecedent,
                        "Consequent": consequent,
                        "Confidence": confidence,
                        "Support": frequent_itemsets[itemset]})
rules_df = pd.DataFrame(rules)
rules_df["Lift"] = rules_df.apply(
    lambda row: row["Support"] /(
        frequent_itemsets[row["Antecedent"]] *
        frequent_itemsets[row["Consequent"]]),
    axis=1)
rules_df.head()
ofreq_df = pd.DataFrame({
    "Itemset":[str(list(i)) for i in frequent_itemsets.keys()],
    "Support":list(frequent_itemsets.values())})
top_freq = freq_df.sort_values("Support",ascending=False).head(15)
plt.figure(figsize=(10,6))
plt.barh(top_freq["Itemset"],top_freq["Support"])
plt.xlabel("Support")
plt.title("Top Frequent Itemsets")
plt.gca().invert_yaxis()
plt.show()
plt.figure(figsize=(8,6))
plt.hist(freq_df["Support"],bins=15)
plt.xlabel("Support")
plt.ylabel("Frequency")
plt.title("Support Distribution")
plt.show()
plt.figure(figsize=(8,6))
plt.scatter(rules_df["Support"],rules_df["Confidence"])
plt.xlabel("Support")
plt.ylabel("Confidence")
plt.title("Association Rules")
plt.show()
```

Output

Top 5 Frequent itemsets with support, confidence and lift values are displayed.

	Antecedent	Consequent	Confidence	Support	Lift
0	(Physical_Activity_Hours_Per_Day_Low)	(Sleep_Hours_Per_Day_High)	0.661051	0.3335	1.324751
1	(Sleep_Hours_Per_Day_High)	(Physical_Activity_Hours_Per_Day_Low)	0.668337	0.3335	1.324751
2	(Stress_Level_Moderate)	(Study_Hours_Per_Day_Low)	0.753709	0.2540	1.482221
3	(Physical_Activity_Hours_Per_Day_Low)	(Social_Hours_Per_Day_High)	0.625372	0.3155	1.281499
4	(Social_Hours_Per_Day_High)	(Physical_Activity_Hours_Per_Day_Low)	0.646516	0.3155	1.281499





Conclusion

The Apriori algorithm generated association rules showing relationships between different student lifestyle factors. The output indicates rules with confidence values ranging from about 0.62 to 0.75 and support values around 0.25–0.33, while lift values are greater than 1 (around 1.28–1.48). For example, the rule **(Stress_Level_Moderate → Study_Hours_Per_Day_Low)** has the highest confidence of about 0.75, indicating a strong association. These numerical outputs show that meaningful relationships exist between factors such as physical activity, sleep, social hours, and study hours, confirming that the Apriori algorithm successfully discovered useful patterns in the dataset.

Experiment 4

Title

Implementation of Market Basket Analysis

Aim

To identify relationships between products using association rules.

Learning Objective

- To analyze product purchase patterns.
- To generate association rules.
- To interpret support and confidence.

Theory

Market Basket Analysis is used to find relationships between items purchased together in transactions. It uses association rule mining techniques such as support, confidence, and lift. Support indicates how frequently items appear together. Confidence measures the reliability of the rule. Businesses use this analysis to improve product placement and recommendation systems.

Lab Outcome

Students will be able to generate and analyze association rules from transaction data.

Program

```
from mlxtend.frequent_patterns import association_rules

rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)

print(rules)
```

Output

Association rules with support, confidence, and lift values.

Conclusion

Market Basket Analysis was successfully performed to discover product relationships.

Experiment 5

Title

Reinforcement Learning Implementation

Aim

To implement reward calculation, discounted reward, optimal action, and Q-learning.

Learning Objective

- To understand reinforcement learning concepts.
- To calculate rewards and discounted rewards.
- To implement Q-learning for optimal decision making.

Theory

Reinforcement learning is a machine learning approach where an agent learns by interacting with the environment. The agent takes actions and receives rewards based on the outcomes. The goal is to maximize the cumulative reward over time. Q-learning is a popular reinforcement learning algorithm that updates a Q-table to find the best action in each state. Reinforcement learning is widely used in robotics, gaming, and recommendation systems.

Program

```
import numpy as np
Q = np.zeros((5,5))
alpha = 0.1
gamma = 0.9
reward = 10
print("Reward:", reward)
discounted_reward = reward * gamma
print("Discounted Reward:", discounted_reward)
state = 2
action = np.argmax(Q[state])
print("Optimal Action:", action)
```

Output

```
Reward: 10
Discounted Reward: 9.0
Optimal Action: 0
```

Lab Outcome

Students will understand reward calculation, discounted reward, and optimal action selection.

Conclusion

Reinforcement learning concepts such as reward calculation and optimal action were implemented successfully.

Experiment 6

Title

Time Series Analysis and Forecasting

Aim

To analyze time series data and perform forecasting using ARIMA.

Learning Objective

- To check stationarity of time series data.
- To perform Dickey-Fuller test.
- To generate ARIMA forecasting model.

Theory

Time series analysis deals with analyzing data points collected over time. Stationarity is an important property where the statistical characteristics of the data remain constant over time. If data is non-stationary, transformations such as differencing are used to make it stationary. The Dickey-Fuller test is used to statistically check stationarity. ARIMA models are widely used for time series forecasting.

Program

```
import pandas as pd
from statsmodels.tsa.stattools import adfuller
data = [10,12,13,15,18,20,22,24]
result = adfuller(data)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

Output

Displays ADF statistic and p-value indicating stationarity.

Lab Outcome

Students will be able to analyze and forecast time series data.

Conclusion

Time series analysis techniques were applied to evaluate stationarity and forecasting.

Experiment 7

Boosting

Title

Boosting Techniques using Cross Validation and AdaBoost

Aim

To implement boosting techniques for improving model accuracy.

Learning Objective

- To understand ensemble learning.
- To evaluate models using cross validation.
- To implement AdaBoost classifier.

Theory

Boosting is an ensemble learning technique that combines multiple weak learners to create a strong learner. It improves prediction accuracy by focusing on misclassified instances in previous iterations. AdaBoost is one of the most popular boosting algorithms. Cross validation is used to evaluate the performance of machine learning models. Boosting techniques are widely used in classification and predictive modeling tasks.

Program

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target
model = AdaBoostClassifier()
scores = cross_val_score(model, X, y, cv=5)
print("Cross Validation Scores:", scores)
```

Output

Cross Validation Scores: [0.96 0.93 0.96 0.93 1.00]

Lab Outcome

Students will be able to implement boosting techniques and evaluate model performance.

Conclusion

Boosting techniques such as AdaBoost and cross validation were implemented successfully.